

Advanced Programming for Scientists

Lecture 7

Kubernetes & HTCondor

Orchestrating scientific workflows



v1.0

Prof. Mateusz Bawaj

Motivation

Modern scientific workflows:

- large-scale simulations (HPC, Monte Carlo)
- data pipelines (GW detectors, astronomy surveys)
- ML training (e.g. RL for cavity locking)

Problems:

- reproducibility across environments
- scaling from laptop → cluster
- managing dependencies and services

Solution:

Container orchestration with Kubernetes

From Docker to Kubernetes

Docker:

- single container runtime
- ideal for reproducibility

Limitation:

- does not manage multiple containers across machines

Kubernetes:

- orchestrates containers across clusters
- handles:
 - scheduling
 - scaling
 - fault tolerance

Think of Kubernetes as: an operating system for distributed scientific applications

Core Concepts

Pod

- smallest unit (one or more containers)

Node

- physical or virtual machine

Cluster

- set of nodes

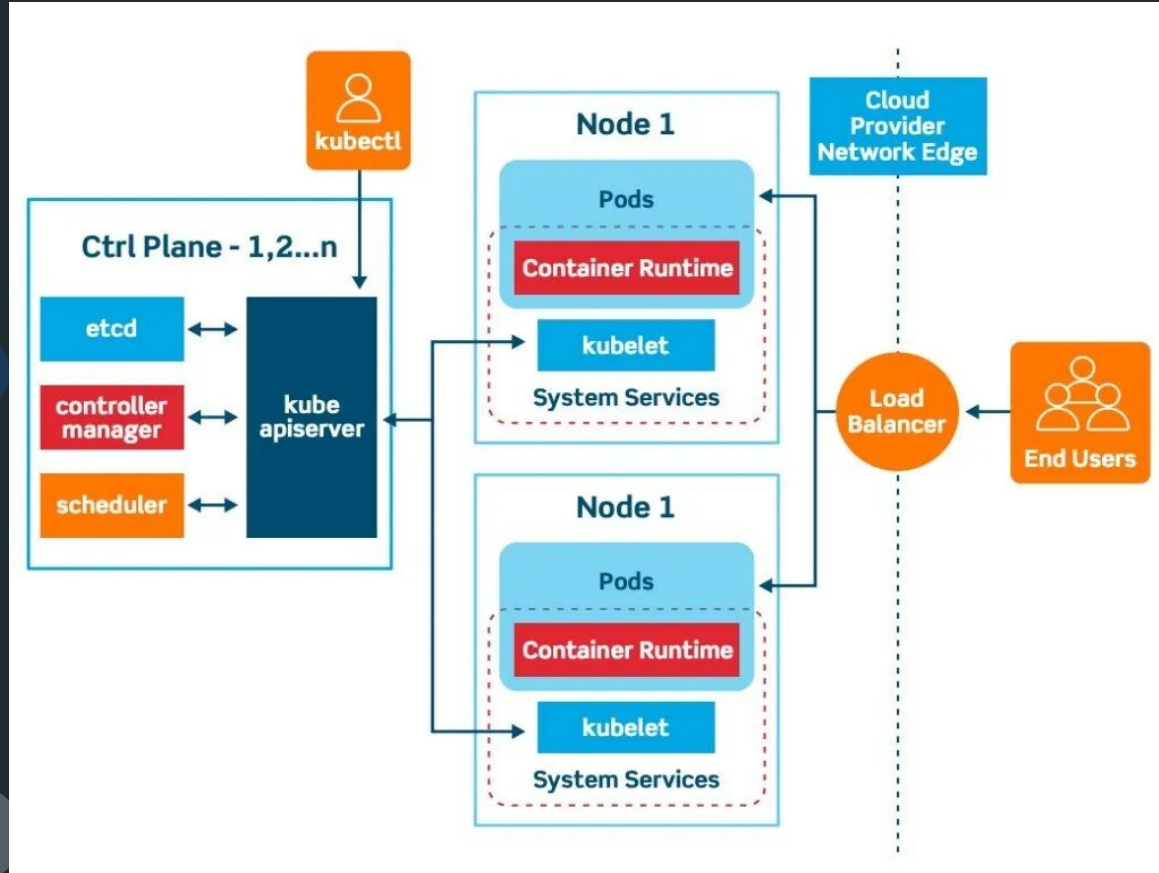
Deployment

- desired state of your application

Service

- exposes your application (networking)

Core Concepts



YAML Configuration (Pod)

Example: simple Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: simulation-pod
spec:
  containers:
  - name: sim
    image: matib12/exercise_legendary-jiggle
  restartPolicy: Never
```

YAML Configuration (Job)

```
apiVersion: batch/v1
kind: Job
metadata:
  name: simulation-job
spec:
  completions: 4
  parallelism: 4
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: sim
        image: matib12/exercise_legendary-jiggle
        imagePullPolicy: Always
```

YAML Configuration (Deployment)

```
apiVersion: batch/v1
```

```
kind: Deployment
```

```
...
```

usually it is not a way to go in scientific application. Deployment is designed to assure continuous operation of created pods.

Why Scientists Should Care

Reproducibility:

- exact environment description compatible with docker

Scalability:

- run your code in parallel

Portability:

- same pipeline on:
 - laptop
 - university cluster
 - cloud

Example: LIGO-Virgo-KAGRA computing resources implement this technology.

I'm sure that other experiments do it as well.

Minikube

Local Kubernetes cluster

```
$ minikube start
```

```
$ minikube dashboard
```

Deploy example:

```
$ kubectl apply -f deployment.yaml
```

```
$ kubectl get pods
```

Made to run locally.

Deployment Scaling Example

```
$ kubectl scale deployment simulation-deployment --replicas=10
```

- instantly scale computation

Scientific use:

- increase resolution
- run more samples
- test parameter space

Why Kubernetes

and why we may not need everything it offers

- Pod crashes → automatically restarted
- Node fails → rescheduled elsewhere

Scientific benefit:

- long simulations survive failures
- robust pipelines for large experiments

Kubernetes vs HTCondor

Feature	Kubernetes	HTCondor
Model	services / containers	batch jobs
Typical use	microservices, pipelines	simulations, HTC
Scheduling	declarative	queue-based
Scientific relevance	workflows	parameter scans, Monte Carlo

htcondor/mini

Minimal deployment for test on a local PC

```
docker run -d --name htcondor-mini -p 9618:9618  
htcondor/mini:latest
```

HTCondor: Simple Job Submission

Submit machine → sends jobs

Scheduler (schedd) → queues jobs

Worker nodes → execute jobs

Workflow:

submit → queue → execute → collect results

Job Description File

HTCondor uses a submit file:

`job.sub`

equivalent to:

- Dockerfile
- Kubernetes
- but focused on jobs, not containers nor services

Minimal Job

```
universe = vanilla
```

```
executable = /bin/sh
```

```
arguments = "-c 'echo Hello from Condor'"
```

```
output = /jobs/out.txt
```

```
error = /jobs/err.txt
```

```
log = /jobs/job.log
```

```
queue
```

what if I need my custom
python module?

Minimal Job (Container-based)

```
universe          = container
container_image = docker://python:3.10

executable = /bin/sh
arguments  = -c "python -c 'print(42)'"

output = output.txt
error   = error.txt
log     = log.txt
queue
```

Explanation of Fields

universe = container

→ run inside container

container_image

→ execution environment (like Docker image)

executable

→ program to run

arguments

→ arguments of the executable run in the command line

Output Handling

- `output` → `stdout`
- `error` → `stderr`
- `log` → job lifecycle

separation of:

- results
- errors
- execution metadata

The `queue` Command

`queue` – submits one job

`queue 10` – submit 10 independent runs

Submitting a Job

```
$ condor_submit job.sub
```

or if using htcondor/mini

```
$ docker exec -it --user submituser condor  
condor_submit /jobs/job.sub
```

Job Monitoring

`condor_q`

`condor_history`

Code	Meaning
I	Idle
R	Running
C	Completed
H	Held
X	Removed

Removing Jobs

`condor_rm <username>` – remove all user's jobs

`condor_rm -all` – the same, remove all user's jobs

`condor_rm 123.0` – remove particular job

Debugging idle Jobs

```
condor_q -long <jobID>
```

```
condor_q -better-analyze <jobID>
```

```
search for Requirements =
```

```
compare with:
```

```
condor_status -long
```

Strengths of HTCondor

- extremely efficient for large number of small jobs
- simple submission model
- robust retry and scheduling

Exercise

Write `job.sub`

Submit 5 jobs:

```
queue 5
```

Monitor:

```
condor_q
```

Inspect results:

```
cat output.txt
```

Key Takeaways

HTCondor = gives access to a **batch computing system**

submit file defines:

- environment
- execution
- outputs

queue enables scaling

monitoring via:

- `condor_q`
- `condor_history`